INFOSYS INTERNSHIP PROGRAM

AI-Powered Intelligent Query Resolution System

A Domain-Agnostic, Voice and Text Enabled Conversational Platform using RAG and Multi-Agent Architecture

WEEK 1 PROGRESS REPORT

Project Understanding, Research and Initial System Design

Details	Information
Submitted by	Team A
Mentor	Vivek Gautam
Reporting period	Week 1
Date of submission	19 Oct 2026

Prepared for the Infosys Internship Program

Table of Contents

S.No.	Title	Page No.
1.	Introduction	3
2.	Week 1 Objectives and Scope	3
3.	Study and Research Summary	3-4
4.	Initial System Architecture	4-5
5.	Proposed Agent Roles and Orchestration	5
6.	Proposed Data Models	5-6
7.	Knowledge Base Ingestion: Design Plan	6
8.	Week 1 Progress and Deliverables	6-7
9.	Challenges and Learnings	7
10.	Plan for Week 2	7
11.	References	7

1. Introduction

1.1 Background

Organizations, institutions and teams maintain information in documents such as handbooks, policies, manuals, FAQs and process guides. Users may find it difficult to locate the right information quickly using manual browsing or keyword search. A conversational query-resolution system can make this information easier to access by allowing users to ask questions in natural language.

1.2 Project Objective

The project aims to design an AI-Powered Intelligent Query Resolution System: a domain-agnostic conversational platform in which users can ask questions about an uploaded knowledge base using text or voice. The proposed system uses Retrieval-Augmented Generation (RAG) and a multi-agent resolution layer. Answers are intended to include relevant source chunks, citations and a retrieval-based confidence indicator.

1.3 Week 1 Focus

Week 1 focuses on understanding the problem statement, studying the foundational technologies, identifying an initial technology stack, and preparing the proposed architecture and workflow. Implementation of the ingestion pipeline and empirical retrieval validation are planned for subsequent work and are not reported here as completed.

2. Week 1 Objectives and Scope

The objectives for the first week are to establish a clear technical direction for the project.

- Understand the project requirements and expected overall capabilities.
- Study the fundamentals of Retrieval-Augmented Generation (RAG).
- Review multi-agent patterns and define preliminary responsibilities for the proposed agents.
- Review browser-based voice input and speech output using the Web Speech API.
- Identify a candidate technology stack for ingestion, embeddings, vector search and API services.
- Prepare an initial layered architecture, orchestration flow and data model.
- Outline the knowledge-base ingestion steps to be implemented and tested in Week 2.

2.1 Scope Boundaries

This report documents research and initial design for Week 1. It does not claim that the ingestion module, multi-agent runtime, voice interface or retrieval evaluation has been implemented or validated. Any technology choices described below are initial design decisions and may be revised after implementation.

3. Study and Research Summary

3.1 Retrieval-Augmented Generation (RAG)

RAG combines information retrieval with language generation. Instead of relying only on knowledge encoded in a language model, the system retrieves relevant passages from an external knowledge base and supplies them as context for generating an answer. This can improve grounding and allow answers to be traced to source documents, provided that the retrieval and citation steps work correctly.

3.2 Main RAG Phases

Phase	Typical activities	Purpose
Indexing (offline)	Load documents, extract text, clean it, split it into chunks, generate embeddings and store vectors with metadata.	Prepare uploaded content for semantic search.

Querying (online)	Encode the question, retrieve relevant chunks, optionally filter or rank them, and provide context to the language model.	Generate a response grounded in retrieved material.

3.3 Initial Design Options

Area	Candidate approach	Reason for considering it
Chunking	Recursive character splitting	Can split long text at paragraph and sentence boundaries while retaining overlap.
Embeddings	sentence-transformers: all-MiniLM-L6-v2	A compact candidate model for local semantic embeddings.
Vector store	ChromaDB	Supports vector search and document metadata storage.
Retrieval	Dense top-k search	Provides a simple baseline before advanced ranking is considered.
API	FastAPI	A Python framework suitable for exposing upload and query endpoints.
Voice	Browser Web Speech API	Can provide speech recognition and speech synthesis where browser support permits.

These are candidate choices from the initial design. They should be confirmed against the implementation requirements, dependency compatibility and the development environment.

3.4 Multi-Agent Design Patterns

- Sequential pipeline: components run in a defined order, which is comparatively straightforward to debug.
- Router pattern: query type determines which processing path is used.
- Clarification: ambiguous questions can trigger a follow-up question before retrieval.
- Conversation context: recent turns can help resolve follow-up references, subject to session management.

3.5 Web Speech API

The browser Web Speech API provides SpeechRecognition for speech-to-text and SpeechSynthesis for text-to-speech. Support varies by browser and device, and microphone permission may be required. A typed-input alternative should remain available so that users can use the system when speech recognition is unsupported or unavailable.

4. Initial System Architecture

The proposed system is divided into four logical layers. This is an initial architecture for implementation, not evidence that all components have already been built.

Layer	Proposed responsibilities	Candidate components
Presentation	Chat interface, document upload, voice controls and source transparency.	HTML/CSS/JavaScript or React; Web Speech API
API	Accept uploads and queries, manage sessions and return responses.	Python with FastAPI
Orchestration	Coordinate query understanding, retrieval, generation, clarification and conversation context.	Python modules and an orchestrator
Knowledge and data	Parse and chunk files, create embeddings, store searchable vectors and keep metadata/logs.	Document parsers, sentence-transformers, ChromaDB, SQLite

4.1 Proposed End-to-End Flow

1. The user uploads a supported document or knowledge-base file.
2. The ingestion pipeline validates the file and extracts its text and metadata.
3. The text is cleaned and divided into chunks; embeddings are generated and indexed.
4. The user submits a question by text or supported voice input.
5. The system interprets the question and retrieves relevant chunks from the selected knowledge base.

6. The response component uses the retrieved context to generate an answer and provide source references.
7. The interface displays the answer and available source information; low-relevance retrieval should be handled cautiously.

5. Proposed Agent Roles and Orchestration

Proposed agent	Responsibility	Expected output
Conversation Memory	Use relevant prior turns to contextualize a follow-up question.	Contextualized query
Query Understanding	Identify intent and classify the query as factual, procedural, comparative or ambiguous.	Intent and route
Retrieval	Search the vector store and select relevant chunks.	Ranked source chunks and scores
Clarification	Ask a focused follow-up when the question is ambiguous.	Clarifying question or refined query
Response Generation	Generate an answer grounded in retrieved content and attach source references.	Answer and citations

5.1 Proposed Routing Logic

Query type	Example	Proposed handling
Factual	What is the minimum attendance requirement?	Retrieve relevant text and produce a direct answer.
Procedural	How do I apply for leave?	Retrieve relevant text and present steps in a clear order.
Comparative	What is the difference between two policies?	Retrieve evidence for each item and compare the supported details.
Ambiguous	What is the policy?	Ask which policy the user means before proceeding.

A retrieval score can be used as a relevance signal, but it should not be described as a calibrated probability that the generated answer is correct. Thresholds and confidence labels should be selected and evaluated during testing.

6. Proposed Data Models

The initial design separates indexed knowledge from conversational state. The following entities are proposed to support source tracking, retrieval and later evaluation.

Entity	Example fields	Purpose
Document	doc_id, filename, file_type, upload_time	Track uploaded files.
Chunk	chunk_id, doc_id, text, page_or_row, chunk_index	Store retrievable text segments and provenance.
Embedding	chunk_id, vector	Associate each chunk with its semantic vector.
Session	session_id, created_at	Group messages within a conversation.
Message	msg_id, session_id, role, text, timestamp	Record conversation turns if persistence is required.
Retrieval Log	query, chunk_ids, scores, answered	Support debugging and future evaluation, subject to privacy requirements.

7. Knowledge Base Ingestion: Design Plan

The ingestion module is a planned implementation task. Its purpose is to convert uploaded files into searchable text chunks with metadata and embeddings.

Step	Planned activity	Expected result
1. Validate	Check supported file type, size and whether content is present.	Accepted file or clear validation error.

2. Parse	Extract text from PDF, DOCX and TXT; read FAQ-style records from CSV.	Text records with source metadata.
3. Clean	Normalize whitespace and handle extraction artefacts where appropriate.	Cleaner text for chunking.
4. Chunk	Split long text into manageable segments while preserving useful context and source positions.	Chunks linked to source records.
5. Embed and index	Generate embeddings and store vectors, text and metadata in a vector store.	Searchable knowledge base.

7.1 Planned File Support

- PDF: extract text and preserve page information where possible.
- DOCX: extract paragraphs and useful structural metadata.
- TXT: read text with appropriate encoding handling.
- CSV: preserve each FAQ question-answer pair as a record where applicable.

Successful ingestion, embedding generation and vector indexing will need to be confirmed by running the implementation and inspecting its output in Week 2.

8. Week 1 Progress and Deliverables

The Week 1 report records the foundational research and design scope. The status labels below are intentionally limited to research and planning; implementation and evaluation are not claimed as complete.

Activity	Week 1 status in this report	Evidence / next action
Project objectives and scope	Documented	Confirm against the official project brief.
RAG and multi-agent research	Covered in research summary	Keep notes or references used during study.
Initial architecture	Proposed	Review and refine as implementation begins.
Candidate technology stack	Identified	Confirm package versions and compatibility.
Ingestion module	Design plan only	Implement and test in Week 2.
Retrieval validation	Not yet reported as completed	Prepare test questions and run evaluation after indexing works.

8.1 Evidence to Add When Available

- Screenshot of the actual project folder structure in VS Code.
- Screenshot or log showing successful parsing and indexing of a sample file.
- Test cases and measured retrieval results once the pipeline runs.

9. Challenges and Learnings

The main technical considerations identified during the initial study are:

- PDF text extraction can vary, particularly with tables, multi-column layouts and scanned documents.
- Chunk size involves a trade-off between preserving context and retrieving focused passages.
- Embedding quality and retrieval settings affect whether relevant passages are returned.
- Source metadata should be retained so responses can be traced back to the original document.
- Voice features depend on browser support and permissions, so text input is an important fallback.
- Retrieval similarity is a relevance signal and should be evaluated rather than treated as proof of answer correctness.

10. Plan for Week 2

The next stage is to move from initial design to a small, testable ingestion and retrieval baseline.

8. Set up and verify the Python environment and project dependencies in VS Code.
9. Implement document loading for the supported formats, beginning with a small sample file.
10. Add text cleaning and chunking while retaining useful source metadata.
11. Generate embeddings using the selected candidate model.
12. Create a persistent vector collection and index the generated chunks.
13. Run a small set of test queries and inspect whether the returned chunks contain relevant source text.
14. Record real errors, fixes and test outcomes; do not report unmeasured performance values.

The exact Week 2 scope may be adjusted based on implementation progress and mentor feedback.

11. References

15. Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.
16. Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP.
17. MDN Web Docs. Web Speech API: Speech Recognition and Speech Synthesis. https://developer.mozilla.org/docs/Web/API/Web_Speech_API
18. Chroma documentation. https://docs.trychroma.com/
19. Sentence Transformers documentation. https://www.sbert.net/
20. FastAPI documentation. https://fastapi.tiangolo.com/

End of Week 1 Progress Report